

MIT Academy of Engineering

Activity Report

Subject: RTOS on ARM Processor Lab

Topic: Mini Project using CMSIS v2

Subject / Course	VSEC: RTOS on ARM Processor Lab
Activity Title	Mini Project using CMSIS v2
Department	E&TC

No.	Student Name	PRN / Enrollment No.
1	Kanak Panchal	202301070135
2	Krushna Yerawar	202301070100
3	Aditi Patil	202301070136

1. Aim / Objective

Design and Implementation of a Real-Time Motion Detection and Response System using STM32, RTOS, and MPU6050.

- Interface MPU6050 using I2C to acquire motion data (acceleration/gyro)
- Implement **multitasking using RTOS with priority scheduling**
- Process motion data to detect events (tilt, vibration, sudden movement)
- Use queue for data transfer between tasks
- Use semaphore for synchronization between tasks
- Control output devices (LED/Buzzer) based on motion conditions

2. Hardware & Software Used

#	Item	Details
1	STM32 F410RBTx	ARM Cortex-M4, 100 MHz, 128 KB Flash, I2C + UART peripherals
2	MPU6050	6-axis IMU (Accelerometer + Gyroscope), I2C address 0x68, $\pm 2g$ range
3	STM32 CUBE IDE	Eclipse-based IDE for STM32 development, code generation and debugging
4	STM32 CUMBEMX	Graphical tool for peripheral and RTOS configuration, HAL code generation
5	FreeRTOS	Real-Time OS kernel via CMSIS-RTOS v2 API; preemptive priority scheduler

3. Brief Theory

This project uses FreeRTOS (via CMSIS-RTOS v2 API) on an STM32F410RBTx ARM Cortex-M4 microcontroller to implement a real-time motion detection system. FreeRTOS provides a preemptive, priority-based scheduler that ensures the highest-priority task always gets CPU time first, making it suitable for time-critical embedded applications. Four concurrent tasks are created using `osThreadNew()`: `SensorTask` (High priority) continuously reads raw accelerometer data from the MPU6050 IMU over I2C at 40 Hz, `LogicTask` (Normal priority) receives this data via a message queue (`osMessageQueue`), applies an exponential low-pass filter, and detects tilt or sudden motion by comparing instantaneous and filtered values against thresholds. When motion is detected, it signals `ControlTask` using a binary semaphore (`osSemaphore`), which in turn drives the onboard LED. This multi-task architecture with queue-based data transfer and semaphore-based synchronization cleanly separates sensing, processing, and actuation, demonstrating core RTOS design principles.

4. Thread / Task Design

List the threads created in your program:

#	Thread Name	Priority	Function / Role
1	SensorTask	osPriorityHigh	Reads raw accelerometer X/Y/Z data from MPU6050 via I2C at 40 Hz and enqueues <code>MotionData_t</code> struct into <code>motionDataQueue</code>
2	LogicTask	osPriorityNormal	Dequeues motion data, applies exponential low-pass filter ($\alpha=0.15$), detects tilt/vibration, and releases semaphore on motion event. Also prints filtered values over UART.
3	ControlTask	osPriorityBelowNormal	Acquires binary semaphore; turns on LED (PA5) for 200 ms when motion is detected, otherwise keeps LED off.

CMSIS-RTOS v2 Primitives Used:

- `osSemaphoreNew()` / `osSemaphoreRelease()` / `osSemaphoreAcquire()` — binary semaphore (`motionSemaphore`) used by `LogicTask` to signal `ControlTask` when motion is detected
- `osMessageQueueNew()` / `osMessageQueuePut()` / `osMessageQueueGet()` — message queue (`motionDataQueue`, depth 5) used to pass `MotionData_t` structs from `SensorTask` to `LogicTask`
- `osThreadNew()` — creates four tasks: `defaultTask`, `SensorTask`, `LogicTask`, `ControlTask` with distinct priorities using the CMSIS-RTOS v2 thread management API
- `osDelay()` — used in `SensorTask` (25 ms, 40 Hz sampling) and `ControlTask` (200 ms LED hold) for periodic timing and non-blocking delays

5. Program Code

Paste your complete program code below. Add brief inline comments.

```
/* -----RTOS CMSIS v2 Mini Project – Motion Detection System -----
Team: Kanak Panchal | Krushna Yerawar | Aditi Patil    Date: 24/04/2026
/* USER CODE END Header */
```

```

/* Includes -----*/
#include "main.h"
#include "cmsis_os.h"
#include <stdio.h>    // For sprintf
#include <string.h>    // For strlen
#include <math.h>      // For fabs
/* Private includes -----*/
/* USER CODE BEGIN Includes */
/* USER CODE END Includes */
/* Private typedef -----*/
/* USER CODE BEGIN PTD */
typedef struct {
    float ax, ay, az;
} MotionData_t;

osSemaphoreId_t motionSemaphoreHandle;
const osSemaphoreAttr_t motionSemaphore_attributes = {
    .name = "motionSemaphore"
};
/* USER CODE END PTD */
/* Definitions for defaultTask */
osThreadId_t defaultTaskHandle;
const osThreadAttr_t defaultTask_attributes = {
    .name = "defaultTask",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityNormal,
};
/* Definitions for SensorTask */
osThreadId_t SensorTaskHandle;
const osThreadAttr_t SensorTask_attributes = {
    .name = "SensorTask",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityHigh,
};
/* Definitions for LogicTask */
osThreadId_t LogicTaskHandle;
const osThreadAttr_t LogicTask_attributes = {
    .name = "LogicTask",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityNormal,
};
/* Definitions for ControlTask */
osThreadId_t ControlTaskHandle;
const osThreadAttr_t ControlTask_attributes = {
    .name = "ControlTask",
    .stack_size = 128 * 4,
    .priority = (osPriority_t) osPriorityBelowNormal,
};

```

```

};
/* Definitions for motionDataQueue */
osMessageQueueId_t motionDataQueueHandle;
const osMessageQueueAttr_t motionDataQueue_attributes = {
    .name = "motionDataQueue"
};
/* USER CODE BEGIN PV */
/* In PD */
#define MPU6050_ADDR 0x68 << 1
#define PWR_MGMT_1    0x6B
#define WHO_AM_I      0x75
#define ACCEL_XOUT_H 0x3B

/* In PV */
char uart_buf[100];
/* USER CODE END PV */
/* USER CODE BEGIN RTOS_MUTEX */
/* add mutexes, ... */
/* USER CODE END RTOS_MUTEX */

/* USER CODE BEGIN RTOS_SEMAPHORES */
/* USER CODE BEGIN RTOS_SEMAPHORES */
motionSemaphoreHandle = osSemaphoreNew(1, 0, &motionSemaphore_attributes);
/* USER CODE END RTOS_SEMAPHORES */

/* USER CODE BEGIN 2 */
// Wake up MPU6050
uint8_t temp_data = 0;
HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, PWR_MGMT_1, 1, &temp_data, 1, 100);
/* USER CODE END 2 */
/* add semaphores, ... */
/* USER CODE END RTOS_SEMAPHORES */

/* USER CODE BEGIN RTOS_TIMERS */
/* start timers, add new ones, ... */
/* USER CODE END RTOS_TIMERS */

/* Create the queue(s) */
/* creation of motionDataQueue */
motionDataQueueHandle = osMessageQueueNew (5, sizeof(MotionData_t),
&motionDataQueue_attributes);
/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
/* USER CODE END RTOS_QUEUES */

/* Create the thread(s) */
/* creation of defaultTask */

```

```
defaultTaskHandle = osThreadNew(StartDefaultTask, NULL,
&defaultTask_attributes);

/* creation of SensorTask */
SensorTaskHandle = osThreadNew(StartSensorTask, NULL, &SensorTask_attributes);

/* creation of LogicTask */
LogicTaskHandle = osThreadNew(StartLogicTask, NULL, &LogicTask_attributes);

/* creation of ControlTask */
ControlTaskHandle = osThreadNew(StartControlTask, NULL,
&ControlTask_attributes);

void StartSensorTask(void *argument) {
    uint8_t raw_data[6];
    MotionData_t data;
    for(;;) {
        if(HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, ACCEL_XOUT_H, 1, raw_data, 6, 100)
== HAL_OK) {
            data.ax = (int16_t)(raw_data[0] << 8 | raw_data[1]) / 16384.0f;
            data.ay = (int16_t)(raw_data[2] << 8 | raw_data[3]) / 16384.0f;
            data.az = (int16_t)(raw_data[4] << 8 | raw_data[5]) / 16384.0f;

            osMessageQueuePut(motionDataQueueHandle, &data, 0, 0);
        }
        osDelay(25); // 40Hz sampling
    }
}
```

6. Output / Result

The system was flashed onto the STM32F410RBTx board and tested with the MPU6050 sensor. When the board was held stationary, the UART terminal displayed near-zero filtered X and Y values (e.g., X:0.01 Y:0.02) and the onboard LED (LD2, PA5) remained OFF. When the board was tilted or shaken, the filtered axis values exceeded the 0.8 g threshold (e.g., X:0.85 Y:0.03) or a sudden spike beyond 0.45 g was detected, causing the LED to turn ON for 200 ms before returning to OFF. The semaphore correctly synchronized LogicTask and ControlTask — no spurious LED activations were observed at rest, confirming correct threshold and filter behaviour.

```

File Edit Log Configuration Control signals View Help
AVG_X:0.82 AVG_Y:-0.04 -> TILTED
AVG_X:0.79 AVG_Y:-0.02 -> TILTED
AVG_X:0.78 AVG_Y:-0.04 -> TILTED
AVG_X:0.74 AVG_Y:-0.07 -> STABLE
AVG_X:0.67 AVG_Y:-0.07 -> STABLE
AVG_X:0.62 AVG_Y:-0.09 -> STABLE
AVG_X:0.83 AVG_Y:0.00 -> SHOCK/VIBE
AVG_X:1.01 AVG_Y:0.07 -> SHOCK/VIBE
AVG_X:1.00 AVG_Y:0.04 -> TILTED
AVG_X:0.87 AVG_Y:0.00 -> SHOCK/VIBE
AVG_X:0.84 AVG_Y:0.01 -> TILTED
AVG_X:0.58 AVG_Y:-0.07 -> SHOCK/VIBE
AVG_X:0.64 AVG_Y:-0.09 -> STABLE
AVG_X:0.82 AVG_Y:-0.06 -> SHOCK/VIBE
AVG_X:0.86 AVG_Y:-0.08 -> TILTED
AVG_X:0.87 AVG_Y:-0.07 -> TILTED
AVG_X:0.73 AVG_Y:-0.09 -> SHOCK/VIBE
AVG_X:0.73 AVG_Y:-0.03 -> STABLE
AVG_X:0.84 AVG_Y:-0.03 -> SHOCK/VIBE
AVG_X:0.78 AVG_Y:-0.07 -> TILTED
AVG_X:0.83 AVG_Y:-0.09 -> TILTED
AVG_X:0.78 AVG_Y:-0.13 -> TILTED
AVG_X:0.72 AVG_Y:-0.11 -> STABLE
AVG_X:0.75 AVG_Y:-0.06 -> STABLE

/dev/ttyACM0 115200-8-N-1 DTR RTS CTS CD DSR RI

```

7. Conclusion

This mini project successfully implemented a real-time motion detection and response system on an STM32F410RBTx microcontroller using FreeRTOS via the CMSIS-RTOS v2 API. Three concurrent tasks — SensorTask, LogicTask, and ControlTask — were created with distinct priorities to separate sensing, processing, and actuation responsibilities. A message queue enabled reliable data transfer of accelerometer readings from SensorTask to LogicTask, while a binary semaphore ensured correct event-driven synchronization between LogicTask and ControlTask for LED control. All objectives from the problem statement were met: I2C-based MPU6050 interfacing, multi-task RTOS design with priority scheduling, queue and semaphore usage, and real-time output actuation. The project provided practical insight into RTOS task design, inter-task communication patterns, and real-time constraints in embedded systems.

8. Marks Rubric (For Evaluator)

#	Criteria	Max Marks
1	Correct CMSIS-RTOS v2 API usage	1
2	Working code / output demonstration	2
3	Clarity of report (aim, theory, conclusion)	5
4	Team work & individual contribution	2
Total		10

Team Signature:

Name: Kanak Panchal
Date: 24/04/26

Evaluated by (Faculty):

Name: _____
Sign & Date: _____